

Special Section on SMI 2021

Real-time skeletonization for sketch-based modeling[☆] Jing Ma^a, Jin Wang^b, Jituo Li^b, Dongliang Zhang^{a,*}^a College of Computer Science, Zhejiang University, Hangzhou, 310058, China^b School of Mechanical Engineering, Zhejiang University, Hangzhou, 310058, China

ARTICLE INFO

Article history:

Received 30 June 2021

Received in revised form 13 October 2021

Accepted 16 November 2021

Available online 23 November 2021

Keywords:

Skeletonization

Sketch-based modeling

Straight skeleton

ABSTRACT

Skeleton creation is an important phase in the character animation pipeline. However, handcrafting skeleton takes extensive labor time and domain knowledge. Automatic skeletonization provides a solution. However, most of the current approaches are far from real-time and lack the flexibility to control the skeleton complexity. In this paper, we present an efficient skeletonization method, which can be seamlessly integrated into the sketch-based modeling process in real-time. The method contains three steps: (i) local sub-skeleton extraction; (ii) sub-skeleton connection; and (iii) global skeleton refinement. Firstly, the local skeleton is extracted from processed polygon stroke and forms a subpart along with the sub-mesh. Then, local sub-skeletons are connected according to the intersecting relationships and modeling sequence of subparts. Lastly, a global refinement method is proposed to give users coarse-to-fine control on the connected skeleton. We demonstrate the effectiveness of our method on a variety of examples created from both novices and professionals.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

There is an increasing need for ready-to-animate models, which takes designers lots of time in shape modeling, skeleton creation, and weight painting. However, manually creating the skeleton is labor-intensive, and requires professional training. Sketch-based tools [1] liberate users from the troublesome of shape modeling, making 3D modeling accessible to novices. There are also lots of work focusing on automatic skeletonization [2–7]. However, they are computation expensive and far from real-time. The skeleton is extracted only after the entire mesh model is constructed and processed by the algorithm.

In this paper, we present a real-time skeletonization method for sketch-based modeling, which enables simultaneous shape modeling and skeleton creation. A local skeleton and corresponding mesh are automatically created as soon as the polygon stroke is captured, then the subpart is connected to the existing subpart, finally a coarse-to-fine refinement strategy is provided to the user to control the skeleton complexity, see Fig. 1. Extracting an animatable skeleton in real-time is difficult with three major challenges: (i) how to extract the local skeleton efficiently from an underlying shape; (ii) how to establish the relationship among

local skeletons in real-time; and (iii) how to find the optimal skeleton structure suitable for animation.

To solve the first challenge, we construct a straight skeleton [8] structure directly from the input polyline and fully utilize Douglas Peucker (DP) simplification [9] to accelerate the process. The straight skeleton is extracted by offsetting the polygon edges inward and recording the trace of vertices. We employ straight skeleton as our base skeletonization technique because the computation cost is almost negligible compared with other methods based on binary image [10], voxels [2,11,12], point clouds [4], and meshes [3,5,7]. The simplicity of polygon data structure makes the algorithm a perfect candidate for immediate skeleton extraction. Our innovation is the full-fledged use of DP technique for speeding up animatable skeleton extraction. As shown in Fig. 1(a), we simplify the dense sketch points (red) with DP, the simplified contour not only approximates the shape well but also generates the most concise input for the straight skeleton extraction algorithm. We also propose an efficient BoundedDP algorithm for extracting joints from the axis line of the straight skeleton. Both the axis curvature and silhouette shape are considered in the simplification, yielding a good result of key joints.

To solve the second challenge, we reuse the skeleton axis and joints extracted from the first step and develop a general cylinder around the axis as well as an inscribed ball around the joint to represent the 3D shape to accelerate the intersection test. Once two 3D shapes are found intersecting with each other, we attach one skeleton to the other according to the modeling sequence. We set the independent subparts as roots, and all follow-up subparts are attached hierarchically according to the intersection relationship and modeling order. The attach position is located

[☆] Only capitalize first word and proper nouns in the title.This article has been certified as Replicable by the Graphics Replicability Stamp Initiative: <http://www.replicabilitystamp.org>.

* Correspondence to: Zhejiang University, China

E-mail address: dzhang@zju.edu.cn (D. Zhang).

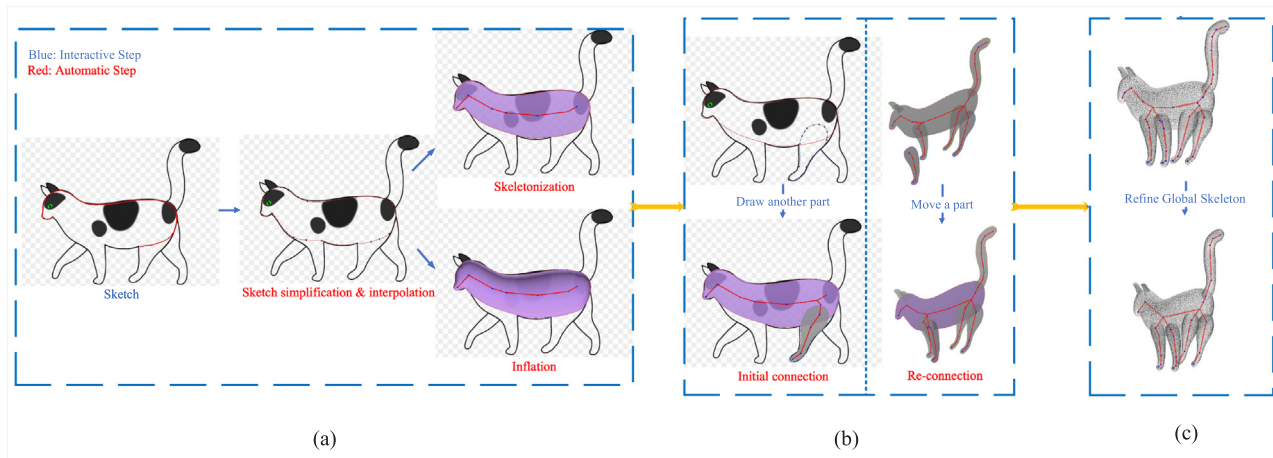


Fig. 1. An illustration of creating animatable skeleton using our system: (a) Draw a sketch, a polygon contour is approximated, and a skeleton is extracted from the simplified polygon, meanwhile, the 3D mesh is created through inflation. (b) Draw/move a part, the hierarchical relationship is deducted by fast shape intersection test and modeling sequence, and the precise connecting position from child part to parent part is calculated. (c) Refine the skeleton complexity by multi-level control operations. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

based on Euclidean distance between parent-skeleton bones and child-skeleton joints.

To solve the third challenge, we equip users with four operations under three-level of detail control. As pointed by [7], animators need multiple-level of control for skeleton complexity, and the fixed number of joints extracted solely from shape topology is not enough to capture user intention. For example, a hand may be represented by a single medial bone or a finer resolution of hierarchical finger joints. Our four operations provide users flexible control over skeleton complexity. Instead of manually inserting, deleting, and connecting bones, users only need to adjust four parameters to change the skeleton structure. The branch-level operation is based on DP algorithm, which allows users to tune the joints number for a single axis, such as the axis on a finger. For the subpart level and global level, we take inspiration from polygon mesh processing [13–15] and skeleton pruning [16,17] since a skeleton can be regarded as an acyclic graph structure (tree). The three operations are *joints merging*, *branch pruning*, and *edge collapsing*. Each operation corresponds to a control threshold that is calculated from the current geometric state of the skeleton. Users can explore different design ideas simply by playing around with these parameters.

In summary, our contributions are four folds: (i) a real-time skeletonization algorithm for immediately constructing an animatable skeleton from user sketch; (ii) an efficient method for fast intersection test and sub-skeletons connection; (iii) a flexible solution for multi-level of details skeleton control; (iv) an easy-to-use sketch system to create animatable models.

2. Related work

2.1. Skeletonization

There are plenty of works for skeletonization. Based on input data types, they can be classified into: binary image and planar curve in 2d; voxels, point cloud, and mesh in 3d [18]. According to the skeletonization technique, they can be classified into propagation-based, and geometric property-based. In the following, we shall discuss these approaches according to the latter categorization.

Propagation-based methods. The propagation-based methods mimic the grassfire or wavefront transformation moving along the boundary, and the skeleton is the quench where two or more

fires/wavefronts meet. In the 2D binary domain, the most representative work is Zhang-Suen Thinning [10], which iteratively removes pixels along object borders until no more pixels could be removed. A large body of follow-up works extend this method to 3D voxel domain [11,19–24]. These methods are not geometric robust and have the risk of removing important features. Despite the additional processing method [25] is proposed to handle excessive removal, the binary propagation approach is generally computation intensive. The computation cost is greatly reduced in the 2D geometric domain, namely, the shape represented by 2D lines or curves. Aichholzer introduces a straight skeleton structure [8] for the polygon. The structure is progressively constructed by shrinking the polygon and recording the trace of moving vertices. Felkel presents a practical implementation for this algorithm using a doubly-linked list and a priority queue, and the time complexity only subjects to the number of polygon vertices. The 3D counterpart of this algorithm is proposed by [3]. Similar to shrinking the polygon into zero-area, Au's algorithm guides the triangular mesh shrinking to zero volume by constrained implicit Laplacian smoothing and defines an edge cost to simplify the collapsed mesh into the curve skeleton. Tagliasacchi et al. [5] further improve Au's algorithm through a detailed analysis of Mean Curvature Flow (MCF) during mesh contraction, and introduce a local remeshing schema to enhance numeric stability. The mesh-based methods are time-consuming incurred by the implicit linear system solving in each contraction iteration, and the generated curve skeleton needs post-processings to be animatable. Compared with a curve skeleton, the straight skeleton structure [8] conforms more to an animatable skeleton. Our skeletonization algorithm benefits from such conformance and the simpleness of the planar polygon. Meanwhile, it does not need an expensive mesh contraction process to create the 3D skeleton.

Geometric property-based methods. The geometric methods strive to find the shape centers by analyzing the translation and rotation property of the inner region towards the boundary. One popular choice is Medial Axis Transformation (MAT). The MAT skeleton is defined as the locus of the centers of all maximally inscribed circles (in the 2D domain) or spheres (in the 3D domain). Plenty of MAT literature exist for different geometric entities: the 2D/3D binary ones [2,26–29], the planar curve [30], the point clouds [31], and the mesh [6]. These methods are computationally intensive, numerically unstable excepting the work for planar curve [30], and are sensitive to boundary perturbations. In addition to the

distance transformation property, rotation traits are used in [4]. The curve skeleton is discovered by finding a rotational symmetry axis (ROSA) for an oriented point cloud. They propose an iterative planar cuts method to find the optimal ROSA plane. This method is extended by [32] to the mesh. They use the ROSA method to find a general cylinder decomposition and partial curve skeleton given a polygon mesh. However, these methods are also time-consuming, and the extracted curve skeleton is not ready for animation.

2.2. Automatic rigging

Automatic rigging creates a ready-to-animate bone skeleton [3] and binds the skeleton to the mesh. The pioneering work of automatic rigging is proposed by [33], which fits a predefined skeleton template to the mesh, and calculates the skinning weight through heat diffusion. However, their method is limited by the skeleton template and fails to cope with various shapes. On the contrary, our work is highly adaptable to various shapes in a creating-on-the-go fashion. It is closely related to RigMesh proposed by [34], in which shape modeling and skeleton creating are handled simultaneously. RigMesh uses Constrained Delaunay Triangulation to decompose the silhouette into needle-like triangles and constructs the skeleton by connecting triangle centers. It defines three strategies for skeleton connection: splitting, snapping, and connecting. When a user drags one part close to the other, a connecting suggestion from these three cases is made. The underlying meshes are merged immediately after the user accepts the recommendation. This strategy is unintuitive and inaccurate, leading to the attached mesh deviating from its original position. In contrast, our algorithm can calculate the exact attach position conforming to the user's original intention. With the rapid development of deep learning techniques, the recent work uses a deep neural network to solve the automatic rigging problem [7], however, their approach is limited to the training set and does not work well for arbitrary shapes, and the method also has a high requirement for devices. Our algorithm works for the novel shapes that a network is never trained before, and it is also practical to be deployed in low-end devices.

2.3. Sketch modeling

Sketch-based system pioneered by Igarashi et al. [1] greatly reduces the workload of shape modeling. Users only need to draw few strokes to create a 3D shape. And the subsequent works [35–42] make sketch modeling more complete, with the power of creating complex shapes. The recent work of MonsterMash [43] propose a framework for casual sketch modeling and animation prototyping under single-view, the work provides lots of fun to novice users, however, the strength of simple interaction is also a weakness of the system, especially for professional users who want an accurate model. Our system allows users to create accurate shapes under different views without losing the interaction simplicity.

3. Our method

3.1. Local sub-skeleton extraction

Previous works extract skeleton from the merged mesh [3,5], which is slow in interactive scenarios. To achieve real-time performance, we generate the local sub-skeleton directly from stroke-input. The local method contains three sub-steps: (i) simple polygon acquisition; (ii) straight-skeleton extraction; (iii) and straight-skeleton simplification, shown in Fig. 2. In this section, we first introduce the definition of simple polygon and straight skeleton. Then, we describe the details of three sub-steps.

Definition 1 (Simple Polygon). A simple polygon is a polygon that does not intersect itself and has no holes. See Fig. 3 for an illustration.

Definition 2 (Straight Skeleton). A straight skeleton is a special partitioning of a polygon into monotone regions traced by a continuous inward offsetting of the contour edges [8]. See Fig. 4 for an illustration.

Simple polygon acquisition. To acquire valid and concise input for the local skeletonization algorithm, the raw sketch line needs to be processed. The sketch line is usually smooth but sometimes may contain noises, see Fig. 19, supplementary data, Appendix A. We smooth out these noises by uniformly discretizing the raw line by a small length, which yields a smooth input for our algorithm, shown in Fig. 2a. Then we apply Douglas–Peucker (DP) algorithm [9] to find a simplified polygon best approximating the shape, shown in Fig. 2b, details of DP are introduced in Section Straight-skeleton simplification. Lastly, we arrange the polygon vertices as counter-clockwise oriented to be ready for the next step. We denote the simple polygon as $\mathcal{P}(\mathcal{V}, \mathcal{E})$ consisting a set of vertices $\mathcal{V} = \{v_i\}$ and edges $\mathcal{E} = \{e_i\}$.

Straight skeleton extraction. Then a straight skeleton $\mathcal{S}(\mathcal{P})$ is extracted from the simple polygon $\mathcal{P}(\mathcal{V}, \mathcal{E})$. The basic idea is to propagate edges inward like setting fire around the border, and the straight skeleton is traced by bisectors of polygon edges. The polygons here refer to the initial silhouette polygon and intermediate offsetting polygons generated by propagation, see Fig. 4. In a formal description, all edges move at the same speed along their respective perpendicular direction. During propagation, two types of events may changes polygon's topology: (i) the edges e_i and e_k (adjacent to the edge e_j) collide, and e_j vanishes; (ii) an edge e_k collides with two consecutive edges e_i and e_j , splitting e_k into two edges on collision position. We define the former as *Edge Event* and the latter as *Split Event* following the same convention from [44,45], see Fig. 4. The method is summarized in Algorithm 3, supplementary data, Appendix A.

Straight-skeleton simplification. The straight skeleton from the previous step contains many unnecessary vertices and edges as shown in Fig. 2c and Fig. 6. Skeleton vertices traced from high curvature borderline are clustering together. Neither the peripheral edges nor the very short skeleton edges are wanted for constructing an animatable skeleton. Thus, we remove all peripheral edges, and collapse short skeleton edges less than a given threshold, shown in Fig. 2d. In our experiment, we set the collapse threshold as $0.5 * \text{average}(\text{skeleton edge length})$, which produces a clean animatable skeleton in most cases. Still, this skeleton is not concise enough. Designers may find that nodes along the long axis are superfluous. The Douglas Peucker algorithm [9] is a good option to do the simplification. However, only considering the axis curve while ignoring the shape will lead to an over-simplified or even out-of-shape skeleton, see Fig. 5. To address this problem, we propose a BoundedDP algorithm considering both the axis and the shape. Details of this algorithm will be introduced in the following.

A spline curve is interpolated for each long polyline, shown in Fig. 2e. We observe that the region enclosing a long branch usually forms a general cylinder, which could be used as boundary restriction during simplification. To extract the general cylinder, we first generate a set of uniformly sampled points $\{p_i\}$ along the axis curve, and compute a segment s_i at each p_i by intersecting the polygon with an infinite line perpendicular to the central axis at p_i , see Fig. 2f. Meanwhile, the intersected edges on the polygon are also recorded, forming two nearly parallel lines on the left and the right side of the central axis. By closing open holes at

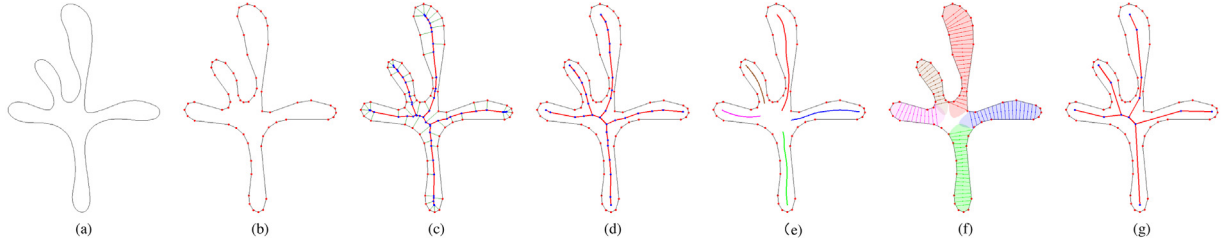


Fig. 2. Local sub-skeleton extraction. (a) Processed sketch line. (b) Simplified polygon. (c) Straight skeleton traced by propagating polygon edges inward. (d) Animatable clean skeleton by removing redundant vertices and edges. (e) Spline interpolated by long polyline branch. (f) Slice uniformly along the spline to capture contour variation. (g) Simplified skeleton by bounded DP simplification. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

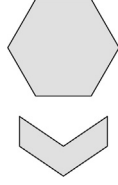


Fig. 3. Simple polygon.

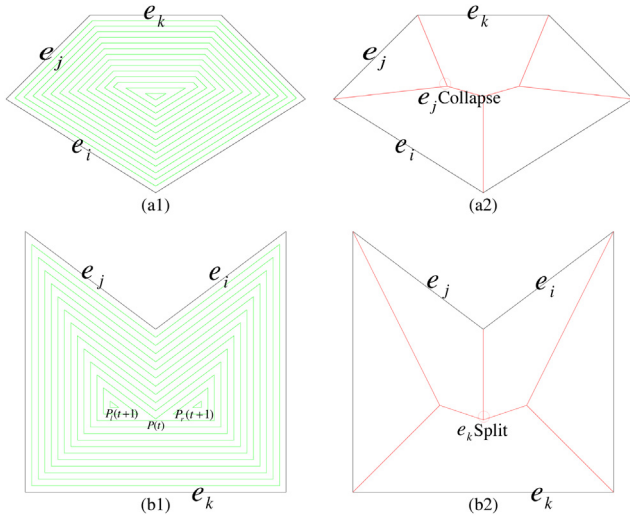


Fig. 4. Black: the initial silhouette polygon. Green: the intermediate offsetting polygons. Red: the traced straight skeleton. (a) Edge event: non-consecutive edges e_i and e_k collide, and e_j collapses. (b) Split event: consecutive edges e_i and e_j collide simultaneously with opposite edge e_k , and e_k splits. The polygon $P(t)$ at the time t splits into two smaller polygons $P_i(t+1)$ and $P_r(t+1)$ at the next time $t+1$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

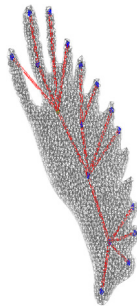


Fig. 5. Over simplification.

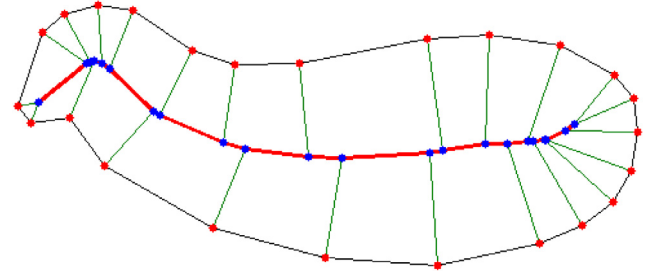


Fig. 6. Straight skeleton. A straight skeleton contains two types of vertices: border vertices (red circle) and skeleton vertices (blue circle), and three types of edges: border edges (black segment), peripheral edges (green segment), and skeleton edges (red segment). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the end of these two lines, we get the general cylinder region Ω , see shadow area in Fig. 2f. Then p_i and Ω are used as input for BoundedDP to get the simplified polyline $l(\{p_i\}, \{e_i\})$ with each polyline edge e_i lying inside the bounded region Ω .

The original Douglas Peucker algorithm works in a recursive greedy manner: at each step, it selects a point p_i with maximum distance to the line $p_s p_e$. If the distance $d_i^\perp$ is larger than a threshold ϵ , p_i is regarded as important for fitting the original line, and is added to the output, see the first row of Fig. 7. The algorithm then recursively calls itself on the split line, $p_s p_i$ and $p_i p_e$, finding the furthest point in each range. The recursion terminates until no point can be found with its distance larger than the threshold ϵ . Two essential ingredients contribute to a good approximation: (i) *the criteria* to select the important point; (ii) *the threshold* to balance between simplicity and approximation quality. For *the criteria*, instead of only considering the variation of axis curve, our algorithm also takes shape variation into account, which leads us to the following formula:

$$E_i = E_{p_i} + \alpha_s E_{s_i} \quad (1)$$

$$= d_i^\perp + \alpha_s (d_i^+ + d_i^-) \quad (2)$$

where E_{p_i} measures the axis variation, namely, the perpendicular distance $d_i^\perp$ from current point p_i to the straight line connecting two endpoints, see the first row of Fig. 7. To capture the shape variation, we add the second term E_{s_i} into the formula, measuring the distances d_i^+ and d_i^- from two endpoints of segment s_i to the two sides of the trapezoid, which is bounded by two end segments s_s and s_e , see the dotted line (distances) and the shaded area (trapezoid) of the second row in Fig. 7. Notice we rearrange the segments $\{s_i\}$ parallelly by stretching the axis curve to a horizontal straight line, filtering the axis variation out of the second term. In each step, we choose the point maximizing error E_i , which is a summation of axis variation and a weighted (α_s)

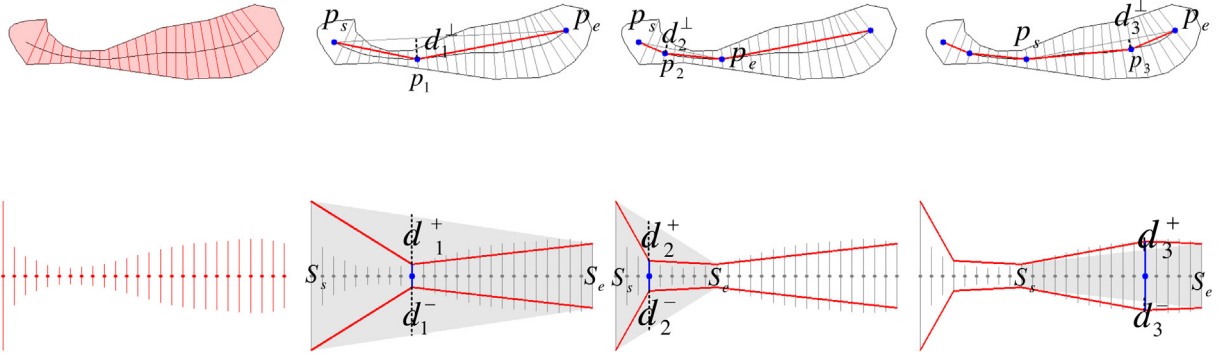


Fig. 7. BoundedDP simplification (red branch in Fig. 2f). The upper part of the first column shows an axis curve and its enclosing general cylinder. The lower part of the first column shows a parallel re-arrangement of intersected segments acquired by uniformly slicing along the axis curve. The next three columns show the first three steps of BoundedDP. Better approximations are found for both the line and the shape as the algorithm progresses.

Input: $\{p_i\}$, $\{s_i\}$, start index i_{st} , end index i_{en} , threshold ϵ
Output: Array of booleans $\{b_i\}$ marking which p_i to be retained.

```

1 if  $i_{en} \leq i_{st} + 1$  then return;
  // Find point with max error
2  $i_{max} \leftarrow i_{st}$ ;
3  $E_{max} \leftarrow 0$ ;
4 for  $i \leftarrow i_{st} + 1$  to  $i_{en}$  do
5    $E_{p_i} \leftarrow \text{PerpendicularDistance}(p_i, \text{Line}(p_{i_{st}}, p_{i_{en}}))$ ;
6    $E_{s_i} \leftarrow d_i^+ + d_i^-$ ; // shape variation, see Fig. 7
7    $E_i \leftarrow E_{p_i} + E_{s_i}$ ;
8   if  $E_i > E_{max}$  then
9      $i_{max} \leftarrow i$ ;
10     $E_{max} \leftarrow E_i$ 
11 end
12 end
13  $b_{i_{max}} \leftarrow \text{true}$ ;
14 if  $E_{max} > \epsilon$  then
15   BoundedDP( $p_i, s_i, i_{st}, i_{max}, b_i$ );
16   BoundedDP( $p_i, s_i, i_{max}, i_{en}, b_i$ );
17 end

```

ALGORITHM 1: BoundedDP: point selection.

shape variation. The point selection procedure is summarized in Algorithm 1. For *threshold*, the larger the value is, the more simplified result we get, sometimes inducing undesirable skeleton, see Fig. 5. To select a proper ϵ , we start with a fairly large initial value ϵ_0 and try to find an approximate polyline $l(\{p_i\}, \{e_i\})$. Then we check whether all edges e_i of the simplified polyline are inside the shape Ω . If there exist intersections between e_i and the shape boundary $\partial\Omega$, we tune down the current threshold ϵ by a factor of 0.8. This procedure is summarized in Algorithm 2.

3.2. Sub-skeleton connection

Before starting this section, we give the following definition for clarity:

Definition 3 (Junction Joint). Joint adjacent to three or more bones.

Definition 4 (Sleeve Joint). Joint adjacent to two bones.

Definition 5 (Terminal Joint). Joint only adjacent to one bone.

Input: n points $\{p_i\}$ and perpendicular segments $\{s_i\}$ uniformly sampled along axis curve, bounded region Ω , a relatively large initial threshold ϵ_0 , adaptive ratio $\alpha \in (0, 1)$
Output: Simplified polyline $l(\{p_i\}, \{e_i\})$

```

1  $\epsilon \leftarrow \epsilon_0$ ; // initial threshold
2 flag  $\leftarrow \text{false}$ ; // is polyline inside shape
3  $l \leftarrow \emptyset$ ;
4 while flag is false do
5    $b \leftarrow \text{Vector}(n, \text{false})$ ;
6   BoundedDP( $p_i, s_i, 1, n, b_i$ );
7   clear previous  $l$ , add  $p_i$  to  $l$  if  $b_i$  is true;
8   flag  $\leftarrow \text{IsInsideShape}(l, \Omega)$ ;
9    $\epsilon \leftarrow \alpha * \epsilon$ ; // reduce threshold
10 end

```

ALGORITHM 2: BoundedDP: threshold tuning.

Definition 6 (Branch). A branch begins with either a junction joint or terminal joint, and ends with a junction joint or terminal joint, probably with sleeve joints between two ends.

We connect sub-skeletons in an interactive pairwise fashion. This conforms to the real scenarios in which users interactively create 3D models part by part. The new subpart is connected to the existing subpart, meanwhile, the new sub-skeleton is connected to the corresponding sub-skeleton. When the user creates a new subpart or moves an existing subpart, see Fig. 1b, we instantly check whether the shape of the current subpart intersects with other subparts. If the current subpart intersects with another subpart, we connect the skeleton in the current subpart to the skeleton in the intersected subpart. To make intersection test real-time, we utilize the local skeleton extracted from the previous step. Specifically, we find a cylinder-ball approximation for the mesh shape, see Fig. 9. Each skeleton edge serves as the medial axis of the general cylinder, and each bone serves as the center of the inscribed ball. The ball's radius is the average distance from the joint to the cross-section contour, which is precomputed by slicing the mesh along the bone's perpendicular direction. Once the radius of the two end joints is computed, the radius for the general cylinder can be calculated by linear interpolation of the two end radius. We denote sub-skeleton on the current subpart as child-skeleton and sub-skeleton on the previous (intersected) subpart as parent-skeleton. Initially, we set the subpart created first as the root. If a subpart does not intersect with any previous subpart, we also add it to the roots set. All subparts created after and intersected with a root becomes the root's child. All subparts created after the child and

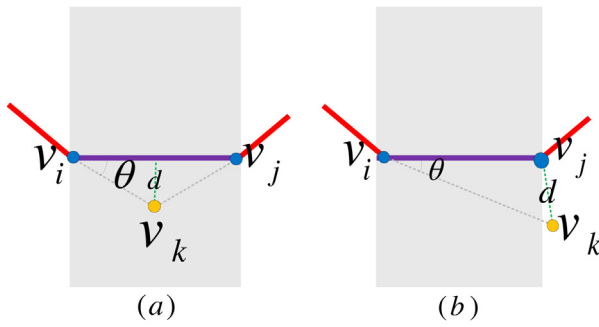


Fig. 8. Distance from child joint (yellow) to parent bone (purple). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

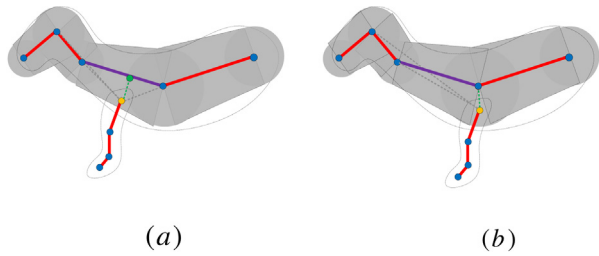


Fig. 9. Sub-skeleton connection: connecting child-leg-skeleton to parent-torso-skeleton. The shadowed area shows the cylinder-ball approximation of the shape, which enables real-time intersection test. The dotted line shows the shortest distance from the candidate child joint to each parent bone. (a) Bone split event. (b) Joint connect event. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

intersected with the child become the child's child (one of the root's descendants), so on and so forth. If the terminal joints or junction joints of the child skeleton lies inside the cylinder-ball region of the parent, we regard that the two parts intersect with each other and start locating a more precise attach position from the child-skeleton to the parent-skeleton.

On child-skeleton, terminal joints and junction joints are selected as the candidates to be connected to the parent bones. Our algorithm then tries to find an optimal position on the parent-skeleton where the child's candidate joints could be connected to, the one minimizing the distance between the child joint and the parent-skeleton. Firstly, we search for the best bone/joint from the parent skeleton. We compute the shortest distance from the child candidate joint to each parent bone. The bone with minimum distance is selected as the best spot for connection, see the purple segment in Fig. 9. Given a parent bone with start joint v_i and end joint v_j , the shortest distance from parent's bone to child's joint v_k is calculated as:

$$d = \begin{cases} |\vec{v_k v_i}| \sin \theta & \text{if } \vec{v_i v_j} \cdot \vec{v_i v_k} \geq 0 \text{ and } \vec{v_j v_k} \cdot \vec{v_j v_i} \geq 0, \\ \min\{|\vec{v_k v_i}|, |\vec{v_k v_j}|\} & \text{otherwise} \end{cases} \quad (3)$$

where $\vec{v_i v_j} \cdot \vec{v_i v_k} \geq 0$ and $\vec{v_j v_k} \cdot \vec{v_j v_i} \geq 0$ defines an influence region for each bone, see shadowed trapezoid area in Figs. 8 and 9. The influence region is bounded by two planes perpendicular to the bone. Only when the child joint lies inside the two planes, the perpendicular point-to-line distance is used Fig. 8a. If child joint v_k is outside the bone's influence region, we calculate the Euclidean Distance from the joint to the bone's two end joints respectively, and choose the smaller one as the shortest distance Fig. 8b.

Then, we connect child-skeleton to parent-skeleton by choosing the closest joint-joint or joint-bone pair. Two types of events

occur: *bone split event* and *joint connect event*, see Fig. 9. *Bone split event* occurs when the child's joint lies inside the parent bone's influence region. We create a new joint on the parent bone, green circle in Fig. 9a, and split the bone into two. Then a new bone is created between the new joint and the child joint. Notice if the child's joint is close to the parent bone's end, one of the split bones will be very short. We address this problem in Section 3.3. *Joint connect event* occurs when the child's joint lies outside the parent bone's influence region. We create a new bone connecting the child joint and the nearest parent joint.

3.3. Global skeleton refinement

Our global refinement method aims to generate a clean skeleton suitable for animation. Specifically, it solves two problems. Firstly, sub-skeleton connection usually induces redundant short bones, especially when two or more child-skeletons connect to similar positions on the parent-skeleton. Secondly, users need more control for skeleton complexity on different parts of the model, for example, users may expect a more complex skeleton for the hand than for the torso. To equip users with interaction easiness and flexibility. We introduce four operations and a multi-level complexity control strategy. The four operations are *curve simplification*, *joints merging*, *branch pruning*, and *edge collapsing*. The multi-level complexity control contains three tiers: *branch level*, *subpart level*, and *global level*. *Simplifying* works on branch level, whereas *merging*, *pruning* and *collapsing* work on subpart level and global level.

Branch level operation. We control branch complexity using BoundedDP of Section 3.1. Specifically, we map the subpart-shape and the branch from 3D space into 2D space by rotating the 3D plane (Figs. 1b, 12 1st row) back to XY plane, run the BoundedDP algorithm in 2D space, and then remap the simplified branch back to its original 3D position. The distance threshold for DP simplification is ϵ_s , and the default value in our experiment is $\epsilon_s = 5.0$. Fig. 10 (green boxes) shows simplifying the wing skeleton by increasing ϵ_s .

Subpart and global level operation. (i) *Joints Merging*: We observe that junction joints usually cluster together after the user connects the child sub-skeleton to the parent sub-skeleton. The distance between any two junction nodes inside the cluster is small, thus it is reasonable to merge these junction nodes into one, see Fig. 11. We perform a breadth-first search to find junction clusters constrained by a distance threshold, it is similar to the algorithm that finds a connected component of graph [46]. The distance threshold for merging is ϵ_m , and we set its default value as $\epsilon_m = 30.0$. (ii) *Branch Pruning*: When a JT-branch (a branch ending with a junction joint and a terminal joint) is shorter than a certain length, we remove it since it is insignificant to the overall structure. The distance threshold for trimming a JT-branch is ϵ_t , and we set the default value as $\epsilon_t = 30.0$. Yellow boxes of Fig. 10 show such an example. (iii) *Edge Collapsing*: Junction joints and sleeve joints are internal joints. When a bone connecting two internal joints is short, it brings redundancy for an animatable skeleton, thus we collapse the two joints into one. The distance threshold for collapsing is ϵ_c , and we set the default value as $\epsilon_c = 10.0$. Black boxes of Fig. 10(b)–(c) show acquiring a more concise skeleton by increasing collapsing threshold ϵ_c .

Multi-level complexity control. We provide the user the option to control the skeleton structure by adjusting the above four parameters ϵ_s , ϵ_m , ϵ_t , and ϵ_c . Since a smaller threshold yields a more complex structure, the skeleton topology changes in a coarse-to-fine manner when users adjust the threshold. For example, a more complex hand skeleton can be acquired by setting

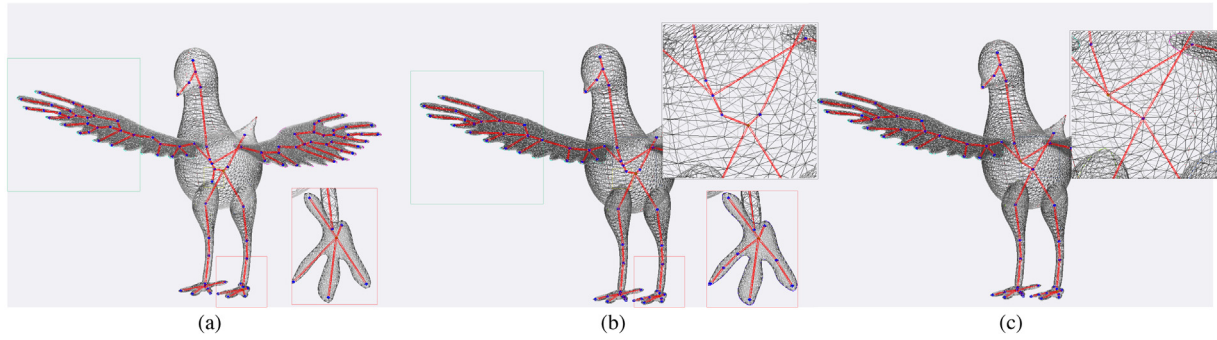


Fig. 10. Refine a bird skeleton. (a) The initial skeleton under default parameters, constructed by our system automatically. Green boxes (a)–(b): simplify branches on wings by increasing threshold ϵ_s . Red boxes (a)–(b): acquire more complex structure on claw by reducing threshold ϵ_s . Yellow boxes (a)–(b): trim short JT branch by increasing threshold ϵ_t . Black boxes (b)–(c): collapse short internal bones by increasing threshold ϵ_c . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

a smaller value of ϵ_s , more knuckles appear on the finger branch, shown in red boxes of Fig. 10a–b. If the user needs the coarse representation, they can adjust the threshold to a higher value. Besides, users can select a single branch, a subpart-skeleton, or the global skeleton, and apply these operations on them. They can explore different skeleton structures simply by playing with these parameters.

4. Results

We implement our approach using C++, libIGL¹ and Qt. The current implementation is tested on a AMD Ryzen 7 3700X 8-Core Processor 3.6 GHz with 16 GB RAM under Ubuntu20.04 LTS system. Our program runs at an interactive rate (see the accompanying video). Fig. 12 shows a gallery of models created by using our system.

4.1. Implementation details

In the first phase of subpart creation, we use CGAL's implementation to extract the straight skeleton [45], which runs in $O(nm + n \log n)$ time, where n denotes the number of polygon vertices and m denotes the reflex ones. For mesh creation, we first triangulate the polygon to get a 2D mesh [47], and generate the 3D mesh by an inflation algorithm [43]. The height field of internal vertices is calculated by Poisson equation subject to the Dirichlet boundary condition and a constant height parameter c . We provide the user the option to change the model's thickness by adjusting the height parameter c . The sparse linear system is solved by LLT Cholesky factorization, which takes $O(n^3)$ time, where n denotes the number of vertices in the 2D mesh. We also provide the user some utility functions such as sketch symmetry line, create symmetry part, rotation, translation, and scaling, etc. After all subparts and the global skeleton are constructed, we merge all parts into a watertight mesh based on the method from [48]. To make our system fully automatic, we use the method from [49] to compute the skinning weights for deformation, see Fig. 12 3rd row.

4.2. Comparisons

We compare our system with MonsterMash² and RigMesh³ to evaluate the interactive part of our system. Twelve users

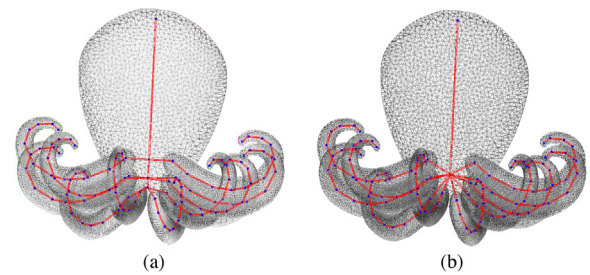


Fig. 11. Merging. (a) Before merging. (b) After merging.

participate in the user study, three with modeling experience are assigned to the professional group, six without any modeling experience are assigned to the novice group, and the rest three are assigned to the jury group to judge the model accuracy (the quality of the mesh and the skeleton). We ask the first two groups to use MonsterMash, RigMesh, and our system to create 3D models, and then show the models to the third group and ask them to fill a questionnaire regarding model accuracy. During the experiment, we give the participant a user manual, and record the time they take to be familiar with the system. On average, it takes eight minutes, twenty minutes, and twelve minutes to train the participants on MonsterMash, RigMesh, and our system respectively. After the user finishes training, they are allowed to proceed to the modeling stage. The modeling time is shown in columns 2–7 of Table 1. Models created by professionals and novices using our system are shown in Figs. 12 and 13 respectively. Models created by professionals using MonsterMash are shown in Fig. 14. Models created by professionals using RigMesh are shown in Fig. 15. The model accuracy given by the jury group is shown in columns 8–13 of Table 1 (MonsterMash models are accessed by mesh quality, RigMesh and our system are accessed by both mesh quality and skeleton quality). Compared with the other two systems, our system takes less time for users to create accurate models.

We compare with other skeletonization methods to evaluate the algorithm part of our system. Our algorithm extracts skeleton from 2D polygon (sub-skeleton extraction), and then transforms the skeleton to the 3D space to construct the 3D skeleton (sub-skeleton connection and global refinement). Therefore, we compare our method with both 2D and 3D skeletonization methods. For 3D skeletonization algorithms, we choose MeanCurvature [5] and GeneralCylinder [32] to compare. MeanCurvature is based on mesh contraction, while GeneralCylinder extracts

¹ <https://libigl.github.io/>.

² <https://monstermash.zone/>.

³ <https://cragl.cs.gmu.edu/rigmesh/>.



Fig. 12. A gallery of models created by professionals using our system (zoom out the PDF to see the details). The first row shows the initial sketches from input image (right corner) and the silhouette polygons (drawn in 2D, then transformed to 3D). The second row shows the refined global skeleton and the inflated 3D mesh. The third row shows the skinning weights (right corner) and the deformed models.

Table 1

Compare our system with MonsterMash and RigMesh. Nov: novices. Pro: professionals. Our system is easy for users to create high-quality models. The easiness is revealed by *Modeling Time* and the quality is revealed by *Model Accuracy*. The shorter the time is, the easier for the user to interact with the system. The more stars are given, the more accurate the model is. The modeling time of our system is half of the RigMesh, while the model accuracy of our system is higher than both MonsterMash and RigMesh.

Models	Modeling time (min)						Model accuracy (1–5 stars)					
	MonsterMash		RigMesh		Ours.		MonsterMash		RigMesh		Ours.	
	Nov	Pro	Nov	Pro	Nov	Pro	Nov	Pro	Nov	Pro	Nov	Pro
Bird	10	6	40	32	18	14	2.5	3.0	3.5	3.5	4.5	4.5
Cat	7	4	29	23	12	10	3.0	3.0	3.5	3.5	4.5	5.0
Crab	5.5	5	33	28	17	15	2.0	2.0	3.0	3.0	4.5	4.5
Deer	5	3.5	42	39	16	13	2.5	3.0	3.0	3.5	4.5	4.5
Tree	8	6	28	21	18	12	3.0	3.5	3.5	3.5	4.0	4.5
Panda	5	3	30	27	16	11	1.5	1.5	2.5	3.5	4.5	4.5
Lorax	3	2.5	21	19	10	8	3.5	3.5	3.0	3.5	4.5	5.0
Octopus	6.5	5	30	22	19	13	1.5	2.0	3.0	3.0	4.5	4.5
Pokemon	4	2	32	24	14	10	2.0	2.0	3.0	3.5	4.5	4.5
Average	6	4.1	31.7	26.1	15.6	11.8	2.4	2.6	3.1	3.4	4.4	4.6

skeleton by slicing mesh to find ROSA [4]. The merged mesh of Fig. 12 is used as input to the 3D skeletonization algorithms. For 2D skeletonization algorithms, we choose Zhang-Suen's Thinning [10] and ChordalAxisTransform of RigMesh [34] to compare. Zhang-Suen's algorithm is based on binary image thinning while ChordalAxisTransform extracts chordal axis from the constrained Delaunay triangulated polygon. We use the subpart contour of

Fig. 12 (1st row) as the input to the 2D skeletonization algorithms. All experiments are conducted in the same machine. Table 2 compares the execution time of our algorithm with others. Our algorithm is significantly faster than MeanCurvature, General-Cylinder, and ZhangSuen's Thinning. We also make qualitative comparisons. The skeleton quality of our algorithm is on par with

Table 2

Execution time (ms) of our algorithm compared with other algorithms. #vertices represents the number of vertices for merged mesh of Fig. 12. Columns 3–4 show the execution time of MeanCurvature and GeneralCylinder respectively. Columns 4–6 show the execution time of Zhang-Suen's thinning [10], RigMesh ChordalAxisTransform [34], and our algorithm. In general, 2D skeletonization is faster than the 3D counterparts. Our method is significantly faster than MeanCurvature, GeneralCylinder and Zhang-Suen's Thinning, and it is slightly slower than ChordalAxisTransform. The relationship between #vertices and the algorithm's time complexity is revealed in Fig. 16.

Models	#vertices	3D Skeletonization		2D Skeletonization		
		MeanCurvature	GeneralCylinder	Zhang-Suen	ChordalAxisTransform	Ours.
Bird	10 808	582	22 052	331	6	13
Cat	6980	598	18 088	167	4	4
Crab	19 079	860	26 382	412	6	10
Deer	10 091	745	39 587	217	4	10
Tree	11 920	681	43 986	141	4	7
Panda	5939	406	14 057	254	6	5
Lorax	8750	719	18 135	212	6	9
Octopus	17 004	1052	21 941	560	5	14
Pokemon	10 487	672	13 195	345	6	7
Average	*	701.8	24 158.7	293.2	5.2	8.8

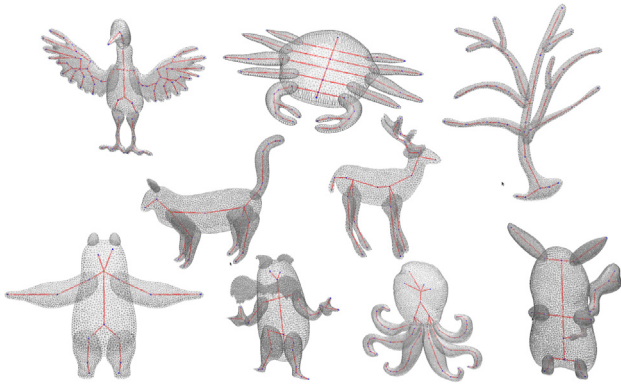


Fig. 13. Models created by novices using our system. Models take an average 15 min to create.

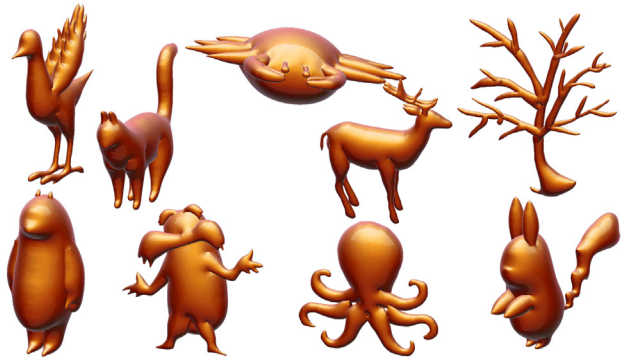


Fig. 14. Models created using MonsterMash [43]. Models take an average 4 min to create. No skeleton is automatically generated. It only supports single-view modeling, which accelerates the modeling time but reduces the shape accuracy, see bird wings, cat ears, and crab body.

MeanCurvature of Fig. 17, and is better than RigMesh of Fig. 15 and GeneralCylinder of Fig. 18.

5. Conclusion and future work

In this paper, we present an efficient framework to create the animatable skeleton in real-time for sketch modeling. We divide the big problem into three smaller ones: (i) local sub-skeleton extraction; (ii) sub-skeleton connection; (iii) global skeleton refinement; and we conquer the problem one by one. To solve the first problem, we extract a straight skeleton by propagating the

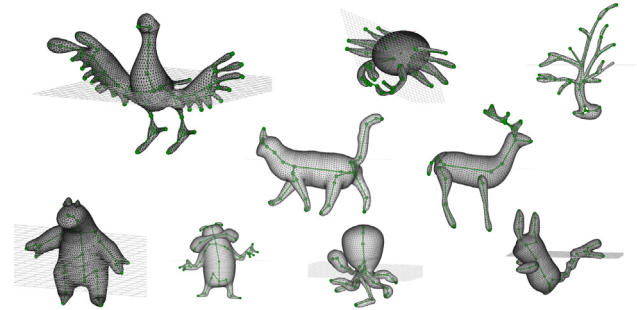


Fig. 15. Models created using RigMesh [34]. Models take an average 26 min to create. The skeleton joints are redundant and inaccurate, see cat butt, panda torso, and lorax shoulder.

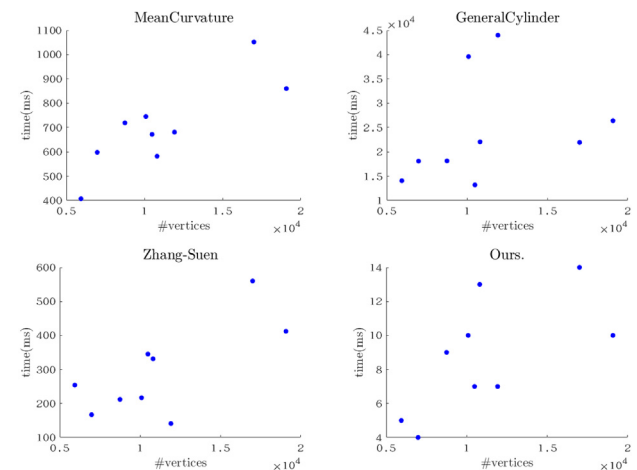


Fig. 16. The execution time is proportional to #vertices.

silhouette polygon inward, and we accelerate the algorithm by finding a concise shape approximation through DP simplification, a novel BoundedDP algorithm is then used to transfer the straight skeleton into the animatable bone skeleton. To solve the second problem of skeleton connection, we abstract the shape with general cylinders for bone and inscribed balls for joints, which enables a real-time intersection test among parts. And the precise attaching position is located by measuring Euclidean distance between parent bones and candidate child joints. To solve the third problem of creating different resolutions of skeletons, we

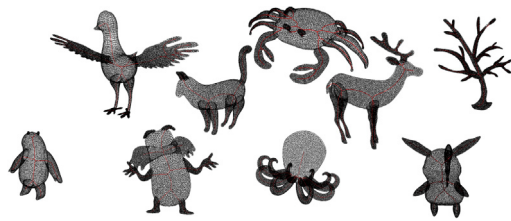


Fig. 17. The curve skeleton extracted by MeanCurvature [5]. The skeleton is smooth and medially centered.



Fig. 18. The disconnected curve skeleton extracted by GeneralCylinder [32]. A color patch represents a general cylinder. The skeleton is sensitive to the general cylinder decomposition, see bird wing, lorax head, and octopus body. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

propose four operations under three tiers of control. The branch-level operation allows users to control the bone complexity of a single axis. The subpart level and global level operation provide users with *joints merging*, *branch pruning*, and *edge collapsing* operations to control the overall structure. The implemented system is demonstrated to be easy to use and enables complex shape & rig creation.

In the future, we intend to provide users more options to create shapes and skeletons by reusing existing 3d models. Specifically, our pipeline will not be limited to the immediately sketched shapes, users can also import 3D models created before and connect them to the newly sketched ones, and our system will automatically create an animatable skeleton for the model consisting of old and new shapes.

CRediT authorship contribution statement

Jing Ma: Conceptualization, Methodology, Software, Writing – original draft, Investigation, Visualization. **Jin Wang:** Resources, Validation. **Jituo Li:** Validation, Investigation. **Dongliang Zhang:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China under 61972340, 61732015 and 51775492.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cag.2021.11.005>.

References

- [1] Igarashi T, Matsuoka S, Tanaka H. Teddy: a sketching interface for 3D freeform design. In: ACM SIGGRAPH 2006 courses. 2006, p. 11–es.
- [2] Coeurjolly D, Montanvert A. Optimal separable algorithms to compute the reverse euclidean distance transformation and discrete medial axis in arbitrary dimension. *IEEE Trans Pattern Anal Mach Intell* 2007;29(3):437–48.
- [3] Au OK-C, Tai C-L, Chu H-K, Cohen-Or D, Lee T-Y. Skeleton extraction by mesh contraction. *ACM Trans Graph* 2008;27(3):1–10.
- [4] Tagliasacchi A, Zhang H, Cohen-Or D. Curve skeleton extraction from incomplete point cloud. In: ACM SIGGRAPH 2009 papers. 2009, p. 1–9.
- [5] Tagliasacchi A, Alhashim I, Olson M, Zhang H. Mean curvature skeletons. In: *Computer graphics forum*, Vol. 31. Wiley Online Library; 2012, p. 1735–44.
- [6] Li P, Wang B, Sun F, Guo X, Zhang C, Wang W. Q-mat: Computing medial axis transform by quadratic error minimization. *ACM Trans Graph* 2015;35(1):1–16.
- [7] Xu Z, Zhou Y, Kalogerakis E, Landreth C, Singh K. Rignet: Neural rigging for articulated characters. 2020, arXiv preprint [arXiv:2005.00559](https://arxiv.org/abs/2005.00559).
- [8] Aichholzer O, Aurenhammer F. Straight skeletons for general polygonal figures in the plane. In: *International computing and combinatorics conference*. Springer; 1996, p. 117–26.
- [9] Douglas DH, Peucker TK. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartogr: Int J Geogr Inf Geovis* 1973;10(2):112–22.
- [10] Zhang TY, Suen CY. A fast parallel algorithm for thinning digital patterns. 1984; 27(3). <http://dx.doi.org/10.1145/357994.358023>.
- [11] Lee T-C, Kashyap RL, Chu C-N. Building skeleton models via 3-D medial surface axis thinning algorithms. *CVGIP: Graph Models Image Process* 1994;56(6):462–78.
- [12] Xu Z, Zhou Y, Kalogerakis E, Singh K. Predicting animation skeletons for 3D articulated models via volumetric nets. In: *2019 international conference on 3D vision (3DV)*. 2019.
- [13] Garland M, Heckbert PS. Surface simplification using quadric error metrics. In: *Proceedings of the 24th annual conference on computer graphics and interactive techniques*. 1997, p. 209–16.
- [14] Botsch M, Kobbelt L. A remeshing approach to multiresolution modeling. In: *Proceedings of the 2004 eurographics/ACM SIGGRAPH symposium on geometry processing*. 2004, p. 185–92.
- [15] Botsch M, Kobbelt L, Pauly M, Alliez P, Lévy B. *Polygon mesh processing*. CRC Press; 2010.
- [16] Montero AS, Lang J. Skeleton pruning by contour approximation and the integer medial axis transform. *Comput Graph* 2012;36(5):477–87.
- [17] Yang C, Indurkha B, See J, Grzegorzec M. Towards automatic skeleton extraction with skeleton grafting. *IEEE Trans Vis Comput Graphics* 2020.
- [18] Tagliasacchi A, Delame T, Spagnuolo M, Amenta N, Telea A. 3D skeletons: A state-of-the-art report. In: *Computer graphics forum*, Vol. 35. Wiley Online Library; 2016, p. 573–97.
- [19] Palágyi K, Kuba A. A hybrid thinning algorithm for 3D medical images. *J Comput Inf Technol* 1998;6(2):149–64.
- [20] Palágyi K, Kuba A. A 3D 6-subiteration thinning algorithm for extracting medial lines. *Pattern Recognit Lett* 1998;19(7):613–27.
- [21] Palágyi K, Kuba A. Directional 3D thinning using 8 subiterations. In: *International conference on discrete geometry for computer imagery*. Springer; 1999, p. 325–36.
- [22] Palágyi K, Kuba A. A parallel 3D 12-subiteration thinning algorithm. *Graph Models Image Process* 1999;61(4):199–221.
- [23] Palágyi K, Balogh E, Kuba A, Halmai C, Erdőhelyi B, Sorantin E, Hausegger K. A sequential 3D thinning algorithm and its medical applications. In: *Biennial international conference on information processing in medical imaging*. Springer; 2001, p. 409–15.
- [24] Ma C-M, Wan S-Y, Lee J-D. Three-dimensional topology preserving reduction on the 4-subfields. *IEEE Trans Pattern Anal Mach Intell* 2002;24(12):1594–605.
- [25] Wang Y-S, Lee T-Y. Curve-skeleton extraction using iterative least squares optimization. *IEEE Trans Vis Comput Graphics* 2008;14(4):926–36.
- [26] Zhou Y, Toga AW. Efficient skeletonization of volumetric objects. *IEEE Trans Vis Comput Graphics* 1999;5(3):196–209.
- [27] Bitter I, Kaufman AE, Sato M. Penalized-distance volumetric skeleton algorithm. *IEEE Trans Vis Comput Graphics* 2001;7(3):195–206.
- [28] Ma W-C, Wu F-C, Ouhyoung M. Skeleton extraction of 3D objects with radial basis functions. In: *2003 shape modeling international*. IEEE; 2003, p. 207–15.

- [29] Hassouna MS, Farag AA. Robust centerline extraction framework using level sets. In: 2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05), Vol. 1. IEEE; 2005, p. 458–65.
- [30] Aichholzer O, Aigner W, Aurenhammer F, Hackl T, Jüttler B, Rabl M. Medial axis computation for planar free-form shapes. *Comput Aided Des* 2009;41(5):339–49.
- [31] Dey TK, Zhao W. Approximate medial axis as a voronoi subcomplex. In: Proceedings of the seventh ACM symposium on solid modeling and applications. 2002; p. 356–66.
- [32] Zhou Y, Yin K, Huang H, Zhang H, Gong M, Cohen-Or D. Generalized cylinder decomposition. *ACM Trans Graph* 2015;34(6):171.
- [33] Baran I, Popović J. Automatic rigging and animation of 3d characters. *ACM Trans Graph* 2007;26(3):72–es.
- [34] Borosán P, Jin M, DeCarlo D, Gingold Y, Nealen A. Rigmesh: automatic rigging for part-based shape modeling and deformation. *ACM Trans Graph* 2012;31(6):1–9.
- [35] Tai C-L, Zhang H, Fong JC-K. Prototype modeling from sketched silhouettes based on convolution surfaces. In: *Computer graphics forum*, Vol. 23. Wiley Online Library; 2004, p. 71–83.
- [36] Cherlin JJ, Samavati F, Sousa MC, Jorge JA. Sketch-based modeling with few strokes. In: Proceedings of the 21st spring conference on computer graphics. 2005; p. 137–45.
- [37] Nealen A, Sorkine O, Alexa M, Cohen-Or D. A sketch-based interface for detail-preserving mesh editing. In: *ACM SIGGRAPH 2005 papers*. 2005, p. 1142–7.
- [38] Schmidt R, Wyvill B, Sousa MC, Jorge JA. Shapeshop: Sketch-based solid modeling with blobtrees. In: *ACM SIGGRAPH 2007 courses*. 2007, p. 43–es.
- [39] Alexe A, Barthe L, Cani MP, Gaildrat V. Shape modeling by sketching using convolution surfaces. In: *ACM SIGGRAPH 2007 courses*. 2007, p. 39–es.
- [40] Nealen A, Igarashi T, Sorkine O, Alexa M. Fibermesh: designing freeform surfaces with 3d curves. In: *ACM SIGGRAPH 2007 papers*. 2007, p. 41–es.
- [41] Sugihara M, De Groot E, Wyvill B, Schmidt R. A sketch-based method to control deformation in a skeletal implicit surface modeler. In: *SBM*. 2008, p. 65–72.
- [42] Cordier F, Seo H, Park J, Noh JY. Sketching of mirror-symmetric shapes. *IEEE Trans Vis Comput Graphics* 2011;17(11):1650–62.
- [43] Dvorožňák M, Sýkora D, Curtis C, Curless B, Sorkine-Hornung O, Salesin D. Monster mash: a single-view approach to casual 3D modeling and animation. *ACM Trans Graph* 2020;39(6):1–12.
- [44] Felkel P, Obdržalek S. Straight skeleton implementation. In: *Proceedings of spring conference on computer graphics*. Citeseer; 1998.
- [45] Cacciola F. A CGAL implementation of the straight skeleton of a simple 2D polygon with holes. In: *2nd CGAL user workshop*, Vol. 1. 2004.
- [46] Hopcroft J, Tarjan R. Algorithm 447: efficient algorithms for graph manipulation. *Commun ACM* 1973;16(6):372–8.
- [47] Shewchuk JR. Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator. In: *Workshop on applied computational geometry*. Springer; 1996, p. 203–22.
- [48] Zhou Q, Grinspun E, Zorin D, Jacobson A. Mesh arrangements for solid geometry. *ACM Trans Graph* 2016;35(4):1–15.
- [49] Jacobson A, Baran I, Popovic J, Sorkine O. Bounded biharmonic weights for real-time deformation. *ACM Trans Graph* 2011;30(4):78.